

Precision Dot Product DV Lab

Final Verification Report

INT8, BF16, and NVFP4 Dot-Product Formal + UVM Verification

Author: Sasha Katne

Project: nvfp4-dotprod-formal-dv

Report date: 2026-07-03

Status: Complete sign-off, public-safe report

This document consolidates the problem statement, DUT architecture, verification strategy, formal properties, regenerated evidence, waveform figures, coverage closure, and conclusions for the mixed-precision dot-product verification project.

Contents

Executive Summary	4
1 Problem Statement and Project Introduction	5
1.1 Problem Statement	5
1.2 Verification Goals	5
1.3 Numeric Modes	5
2 DUT Architecture	7
2.1 Unified Top-Level Datapath	7
2.2 Sequential Wrapper	7
2.3 BF16 Datapath	8
2.4 NVFP4 Datapath	9
3 Verification Plan	10
3.1 Strategy Overview	10
3.2 UVM Scoreboard And Coverage	11
3.3 Bug-Injection Plan	11
4 Formal Properties	12
4.1 INT8 Top Equivalence	12
4.2 Sequential Protocol Properties	12
4.3 BF16 Properties	12
4.4 NVFP4 Properties	13
5 Sign-Off Evidence and Regenerated Waveforms	14
5.1 Formal Results	14
5.2 UVM And Coverage Results	15
5.3 Waveform: Combinational Top Behavior	15
5.4 Waveform: Clean Backpressure Hold	16
5.5 Waveform: Bug-Injected Assertion Failure	16
6 Bug-Injection Traceability	18
7 Conclusion	19
A Reproduction Commands	20

List of Figures

2.1	Unified DUT datapath	7
2.2	Sequential ready/valid wrapper	8
2.3	BF16 datapath and proof decomposition	8
2.4	NVFP4 block-scaled datapath	9
3.1	UVM environment architecture	10
5.1	Combinational top waveform: multi-mode behavior	16
5.2	Sequential wrapper waveform: clean backpressure hold	16
5.3	Sequential wrapper waveform: bug-injected p_hold_stable failure	17

List of Tables

1	Final sign-off snapshot	4
1.1	Numeric tiers verified in the project	5
3.1	Verification methods by target	11
5.1	Unified formal proof results	14
5.2	Final UVM regression tests	15
5.3	Merged coverage closure	15
6.1	Seeded bugs and detection points	18

Executive Summary

This project verifies an 8-lane mixed-precision vector dot-product unit across three arithmetic tiers: INT8, BF16, and NVIDIA-style NVFP4 block-scaled FP4. The verification strategy combines direct formal equivalence, assume-guarantee formal decomposition, directed simulation, UVM constrained-random testing, code and functional coverage, and bug-injection checks.

Table 1: Final sign-off snapshot

Area	Final status
Formal proof jobs	18 total: 11 clean and 7 bug-injected
Clean formal jobs	All target assertions proven
Bug-injected formal jobs	Every mutation falsified at least one intended property
UVM tests	7 tests, zero mismatches, zero leftovers, zero fatal/errors
Coverage	100.00% reachable merged coverage
Waivers	One structurally unreachable branch leg, documented and justified

The central verification principle is a single-source-of-truth golden model: SystemVerilog reference functions in `ref/` are used by both formal properties and simulation scoreboards. This avoids the common verification failure mode where a formal checker and a simulation checker silently validate against different reference behavior.

Raw tool logs, waveform databases, UCDB files, and transcripts are generated artifacts. They are not checked into the repository; this report includes summarized evidence and curated figures only.

1 Problem Statement and Project Introduction

1.1 Problem Statement

Modern accelerator datapaths increasingly mix integer and compressed floating formats in the same compute block. A dot-product unit is deceptively small, but it is verification-dense: sign handling, exact accumulation, final rounding, special values, packed element formats, block scaling, ready/valid protocol, and coverage closure all interact.

The problem addressed by this project is:

Prove and test that a unified vector dot-product design computes correct results for INT8, BF16, and NVFP4 modes, while preserving a correct streaming ready/valid interface and demonstrating that the verification environment catches representative injected bugs.

1.2 Verification Goals

- Prove arithmetic equivalence for INT8 exhaustively.
- Verify the sequential wrapper’s ready/valid and backpressure behavior.
- Prove BF16 behavior using a decomposition that avoids monolithic nonlinear equivalence bottlenecks.
- Prove NVFP4 behavior using pure-DUT-net top assertions and standalone lane, scale, and final-round proofs.
- Use the same reference functions from formal and UVM scoreboards.
- Demonstrate proof strength with bug-injected falsification runs.
- Close reachable coverage to 100.00% and document any unreachable logic.

1.3 Numeric Modes

Table 1.1: Numeric tiers verified in the project

Mode	Computation	Verification emphasis
INT8	Eight signed 8-bit products summed exactly and cast to a 32-bit output.	Exhaustive top equivalence; saturation shown structurally unreachable for one 8-lane dot product.

Mode	Computation	Verification emphasis
BF16	Eight BF16 products accumulated exactly in a constrained fixed-point window, then rounded once to FP32.	Lane decode/product, alignment, special case ladder, and final round proven in separate formal pieces.
NVFP4	Sixteen E2M1 elements per vector share UE4M3 block scales; result is FP32.	Lane product, scale multiply, inner sum, and final normalization verified with assume-guarantee composition.

2 DUT Architecture

2.1 Unified Top-Level Datapath

The top-level DUT exposes widened 16-bit operand lanes and selects the active numeric tier using mode. INT8 consumes the low byte; BF16 consumes the full 16-bit lane; NVFP4 interprets the operand lanes as a packed block of sixteen 4-bit elements plus UE4M3 scale bytes.

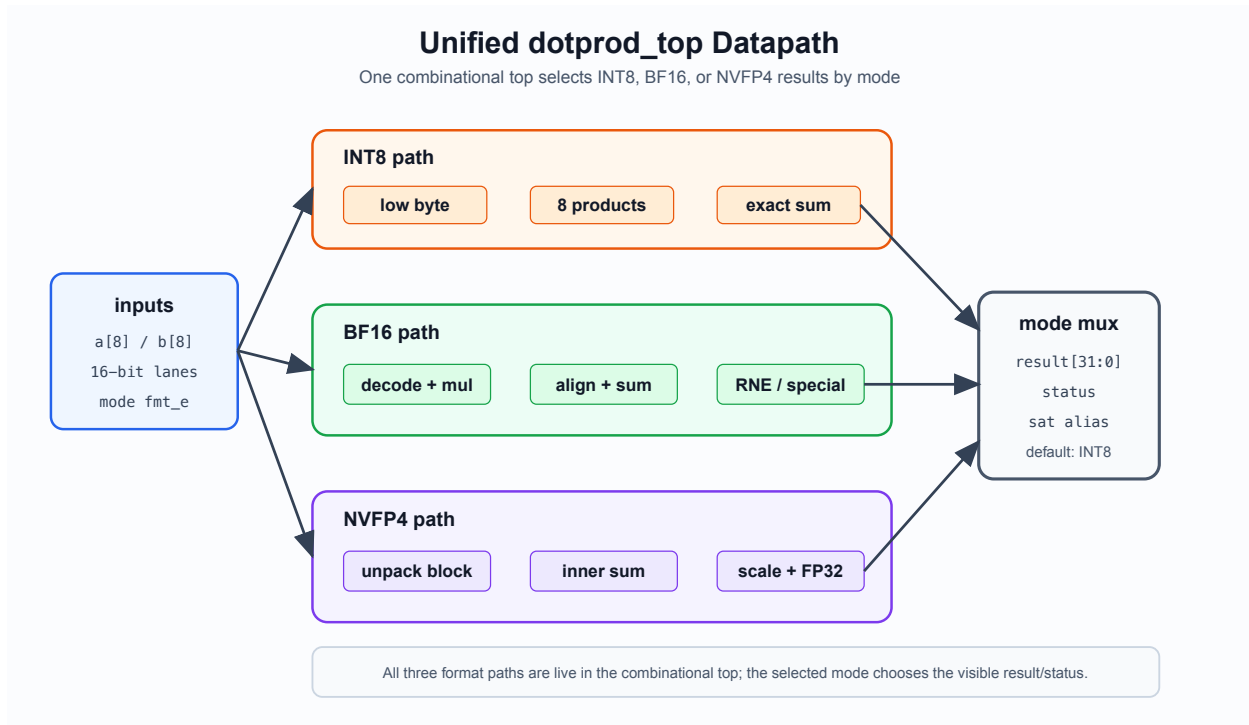


Figure 2.1: Unified DUT datapath

The output port is 32 bits. In INT8 mode it carries the sign-extended integer result. In BF16 and NVFP4 modes it carries an IEEE binary32 bit pattern. Status fields carry saturation, invalid, NaN, and infinity indicators; the legacy sat output aliases status.sat.

2.2 Sequential Wrapper

The sequential wrapper adds a two-stage ready/valid pipeline around the combinational core. A global stall freezes the pipeline whenever a valid output is present and the consumer deasserts out_ready. The key protocol contract is that a held output must remain valid and stable under backpressure.

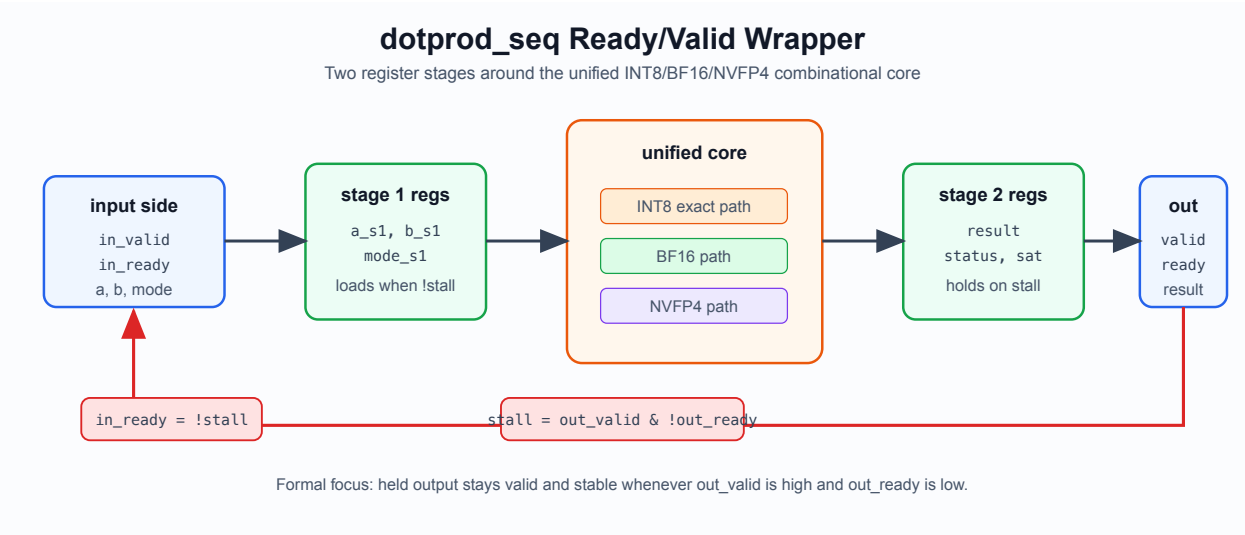


Figure 2.2: Sequential ready/valid wrapper

2.3 BF16 Datapath

The BF16 datapath decodes operands, flushes subnormal inputs to zero, multiplies lanes, aligns products into a fixed-point accumulator window, resolves IEEE special cases, and performs one final round-nearest ties-to-even conversion to FP32.

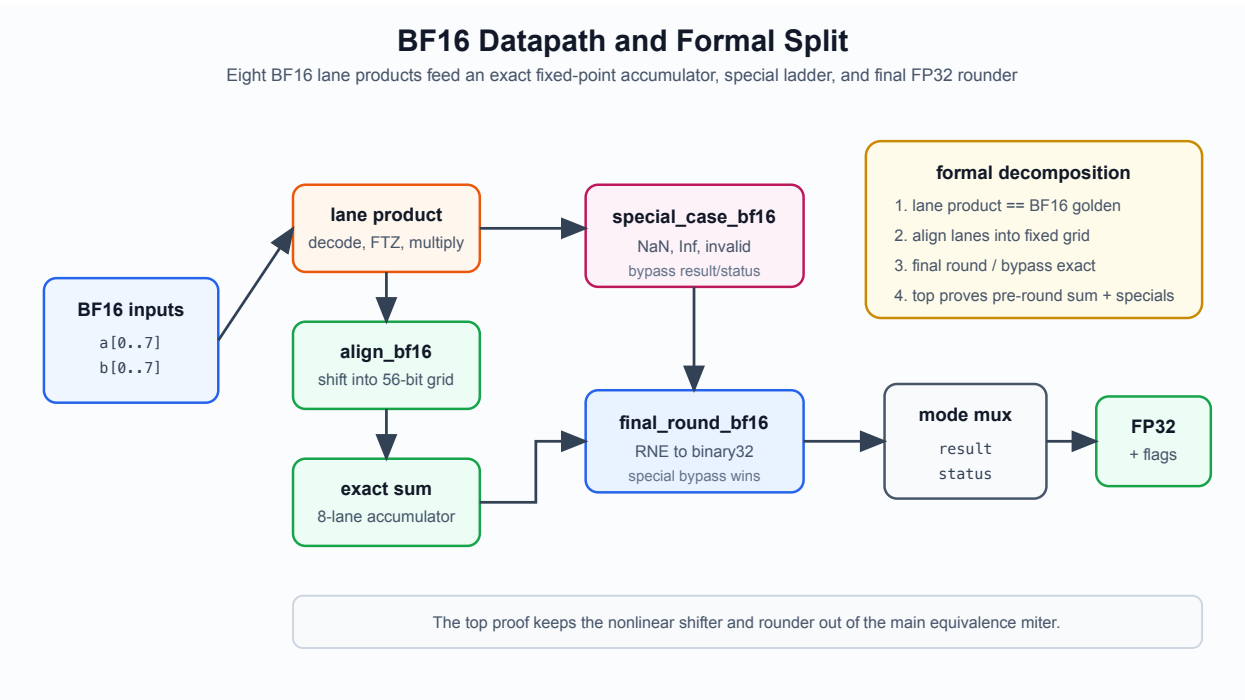


Figure 2.3: BF16 datapath and proof decomposition

2.4 NVFP4 Datapath

NVFP4 mode factors the block-scaled dot product into an exact inner sum and a scale product:

$$\sum_i (a_i s_A)(b_i s_B) = (s_A s_B) \sum_i (a_i b_i)$$

The inner E2M1 product sum is exact, the UE4M3 scale product is checked separately, and final normalization produces FP32.

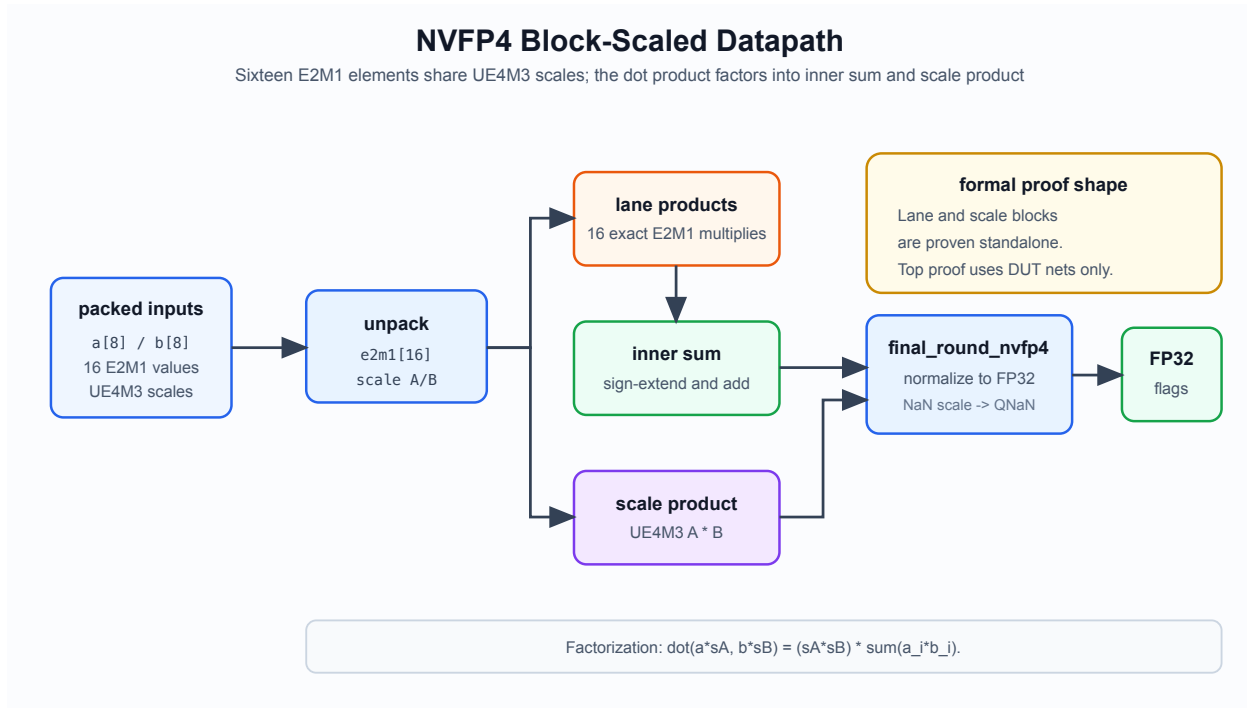


Figure 2.4: NVFP4 block-scaled datapath

3 Verification Plan

3.1 Strategy Overview

The verification plan intentionally uses different proof shapes for different logic classes. Linear INT8 arithmetic is handled as a direct exhaustive equivalence proof. Nonlinear BF16 and NVFP4 logic is split into smaller proofs that can converge independently. The sequential wrapper is verified with protocol SVA and UVM stress tests.

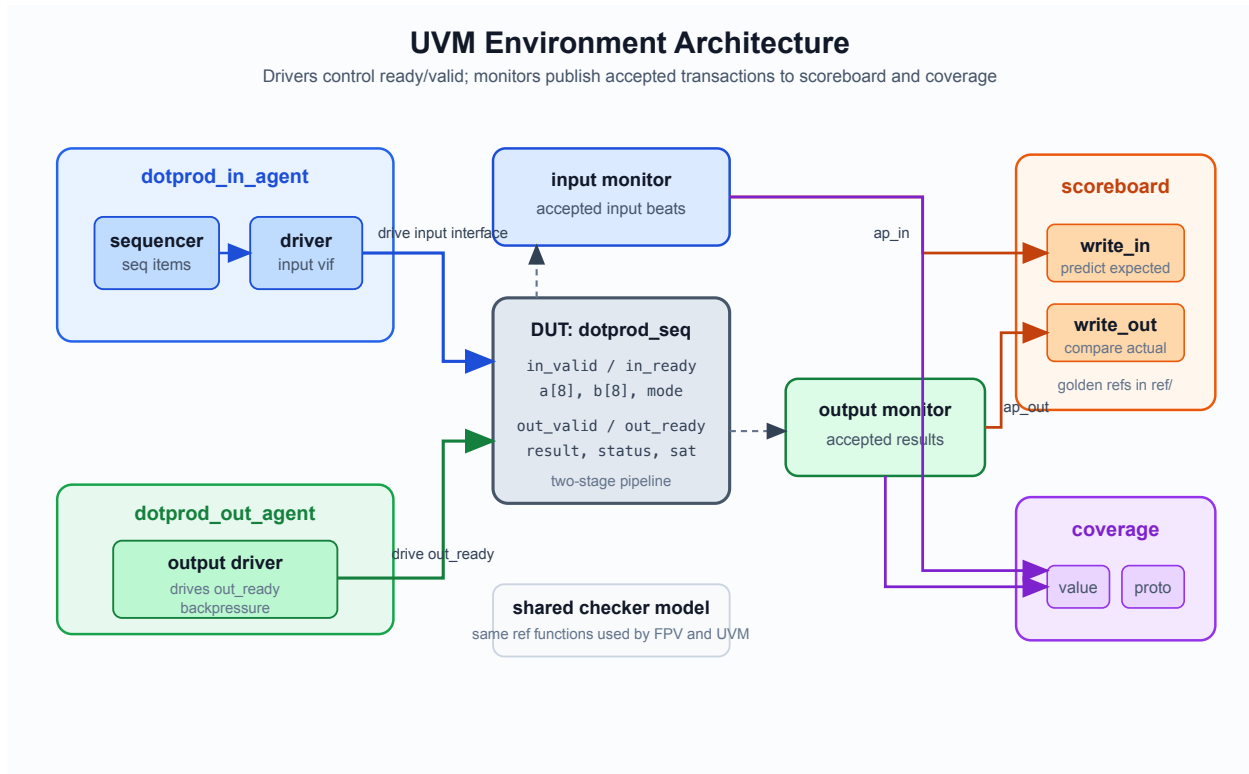


Figure 3.1: UVM environment architecture

Table 3.1: Verification methods by target

Target	Formal plan	Simulation plan
INT8 top	Direct top equivalence against dotprod_ref.	Directed corner tests plus UVM random/corner tests.
Sequential wrapper	Ready/valid protocol assertions.	Random backpressure and deterministic waveform case.
BF16	Lane, align, round, special, and top assume-guarantee proofs.	BF16 random and directed special-value tests.
NVFP4	Lane, scale, final round, and pure-DUT-net top proof.	NVFP4 random and directed block-scale tests.

3.2 UVM Scoreboard And Coverage

The UVM environment uses input and output agents around the ready/valid wrapper. The scoreboard queues accepted input transactions, calls the appropriate golden reference based on mode, and compares each accepted output in order. Coverage tracks value classes, protocol states, BF16 operand/result classes, and NVFP4 element/scale/result classes.

3.3 Bug-Injection Plan

Bug-injection runs are not optional smoke tests; they are the proof-suite sensitivity check. Each mutation is expected to falsify a specific property. A clean proof without a corresponding falsification can still be vacuous or too weak, so this project pairs clean proof runs with targeted failing variants.

4 Formal Properties

4.1 INT8 Top Equivalence

The INT8 proof constrains the top to INT8 mode, evaluates the low byte of each operand lane with `dotprod_ref`, and asserts that result, status, and the sat alias all match the reference.

Listing 4.1: INT8 top equivalence property

```
am_int8_mode: assume (mode == FMT_INT8);

a_result_matches_ref: assert (
    result == FP32_W'(ref_r) &&
    status.sat == ref_sat &&
    sat == ref_sat &&
    !status.invalid && !status.is_nan && !status.is_inf);

c_int8_sat_unreachable: cover (status.sat == 1'b1);
c_positive_result:      cover ($signed(result) > 0);
c_negative_result:      cover ($signed(result) < 0);
```

4.2 Sequential Protocol Properties

Listing 4.2: Sequential ready/valid protocol properties

```
p_hold_stable: assert property
    (out_valid && !out_ready |=>
        out_valid && $stable(result) && $stable(status) && $stable(sat));

p_no_out_at_reset: assert property
    ($rose(rst_n) |-> !out_valid);

p_stall_blocks_ready: assert property
    (out_valid && !out_ready |-> !in_ready);

p_sat_alias: assert property (sat == status.sat);

c_backpressure: cover property (out_valid && !out_ready);
c_accept:      cover property (in_valid && in_ready);
```

4.3 BF16 Properties

BF16 is decomposed into lane, align, round, special ladder, and top assume-guarantee checks. The top proof blackboxes the lane multiplier, assumes the standalone lane guarantee, and proves linear pre-round reduction plus special case equivalence.

Listing 4.3: BF16 top assume-guarantee properties

```

am_bf16_mode: assume (mode == FMT_BF16);

a_acc_is_lane_sum:      assert (sum_bf16 == lane_sum);
a_special_valid_matches_ref: assert (special_valid == ref_pr.special_valid);
a_special_result_matches_ref: assert (special_result == ref_pr.special_result);
a_special_status_matches_ref: assert (special_status == ref_pr.special_status);

c_numeric: cover (!ref_pr.special_valid);
c_special: cover (ref_pr.special_valid);
c_nan:     cover (ref_pr.special_status.is_nan);
c_invalid: cover (ref_pr.special_status.invalid);

```

4.4 NVFP4 Properties

The NVFP4 top proof avoids placing a 16-lane golden multiply inside the hardest miter. Instead, it assumes the independently proven lane guarantee and asserts only over DUT nets for alignment and accumulation.

Listing 4.4: NVFP4 top assume-guarantee properties

```

am_lane_ref:  assume (nvfp4_prod[gi] == ref_mul_nvfp4(nv_ea[gi], nv_eb[gi]));
a_align_lane: assert (nvfp4_wide[gi] == NVFP4_INNER_W'(nvfp4_prod[gi].prod));

am_nvfp4_mode: assume (mode == FMT_NVFP4);
a_inner_is_lane_sum: assert (nvfp4_inner_sum == lane_sum);
a_scale_sig_ref:   assert (nvfp4_scale_sig == ref_pr.scale_sig);
a_scale_exp_ref:   assert (nvfp4_scale_exp == ref_pr.scale_exp);
a_scale_nan_ref:   assert (nvfp4_scale_is_nan == ref_pr.scale_is_nan);

```

The final-round proof checks both normal numeric behavior and NaN-scale bypass:

Listing 4.5: NVFP4 final-round properties

```

a_numeric_round: assert (scale_is_nan ||
    (result == ref_result && status == ref_status));

a_nan_bypass: assert (!scale_is_nan ||
    (result == FP32_QNAN && status.invalid && status.is_nan &&
    !status.is_inf && !status.sat));

```

5 Sign-Off Evidence and Regenerated Waveforms

5.1 Formal Results

The formal, UVM, and coverage tables consolidate the M5 sign-off record preserved in the repository documentation. The timing figures in this chapter were regenerated with report-specific self-checking testbenches so the PDF contains curated waveform evidence without checking raw VCD or transcript artifacts into source control.

Table 5.1: Unified formal proof results

Proof job	Tier	Clean outcome	Bug outcome
fpv_run_top	INT8	1 assert proven	Bug falsifies result equivalence
fpv_run_seq	INT8 wrapper	4 asserts proven	Bug falsifies p_hold_stable
fpv_run_lane_bf16	BF16	21 asserts proven	Lane mutation falsifies
fpv_run_align_bf16	BF16	8 asserts proven	Align mutation falsifies
fpv_run_round_bf16	BF16	2 asserts proven	Round mutation falsifies
fpv_run_special_bf16	BF16	5 asserts proven	Covered by top special mutation
fpv_run_bf16_top	BF16	4 asserts proven	Special mutation falsifies
fpv_run_lane_nvfp4	NVFP4	5 asserts proven	Lane mutation falsifies
fpv_run_scale_nvfp4	NVFP4	3 asserts proven	Scale mutation falsifies
fpv_run_round_nvfp4	NVFP4	2 asserts proven	NaN-bypass mutation falsifies
fpv_run_nvfp4_top	NVFP4	20 asserts proven	NaN-detect mutation falsifies

5.2 UVM And Coverage Results

Table 5.2: Final UVM regression tests

Test	Items	Matched	Mismatched	Leftover
dotprod_random_test	500	500	0	0
dotprod_backpressure_test	1000	1000	0	0
dotprod_corner_test	8	8	0	0
dotprod_bf16_test	500	500	0	0
dotprod_bf16_corner_test	8	8	0	0
dotprod_nvfp4_test	500	500	0	0
dotprod_nvfp4_corner_test	8	8	0	0

Table 5.3: Merged coverage closure

Metric	Result
Statements	100%
Branches	100%
Expressions	100%
Covergroups	100%
Covergroup bins	127/127
Total	100.00%

5.3 Waveform: Combinational Top Behavior

Figure 5.1 shows the report-specific deterministic top waveform. The sequence exercises INT8, BF16, NVFP4 numeric, and NVFP4 NaN-scale cases. The DUT result tracks the expected result from the same golden reference functions used by the scoreboards.

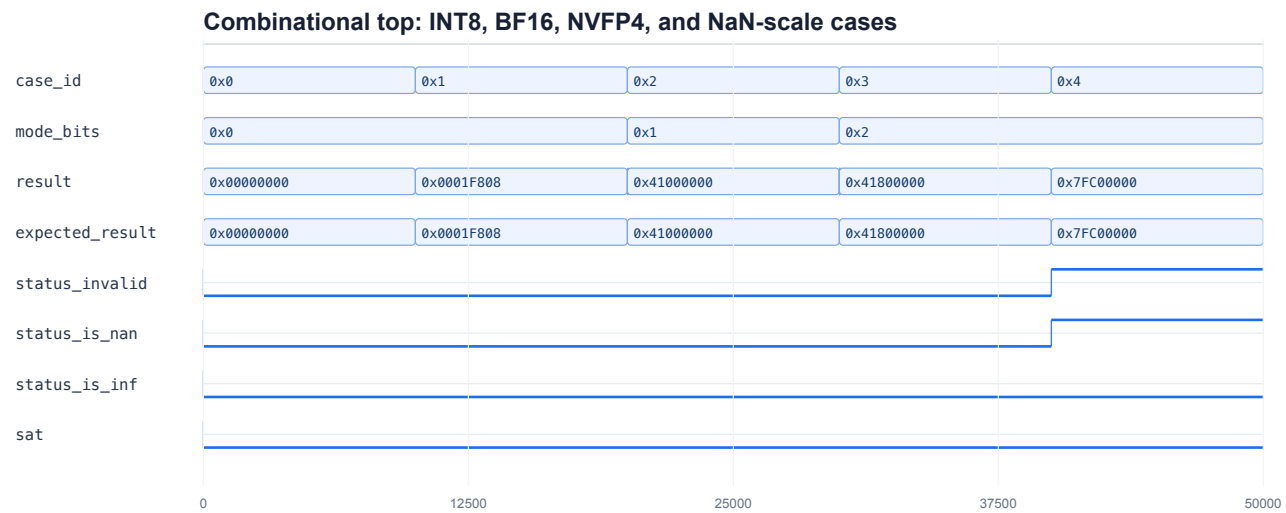


Figure 5.1: Combinational top waveform: multi-mode behavior

5.4 Waveform: Clean Backpressure Hold

Figure 5.2 shows the sequential wrapper holding output valid and result stable while out_ready is low. The report assertion flag remains low.

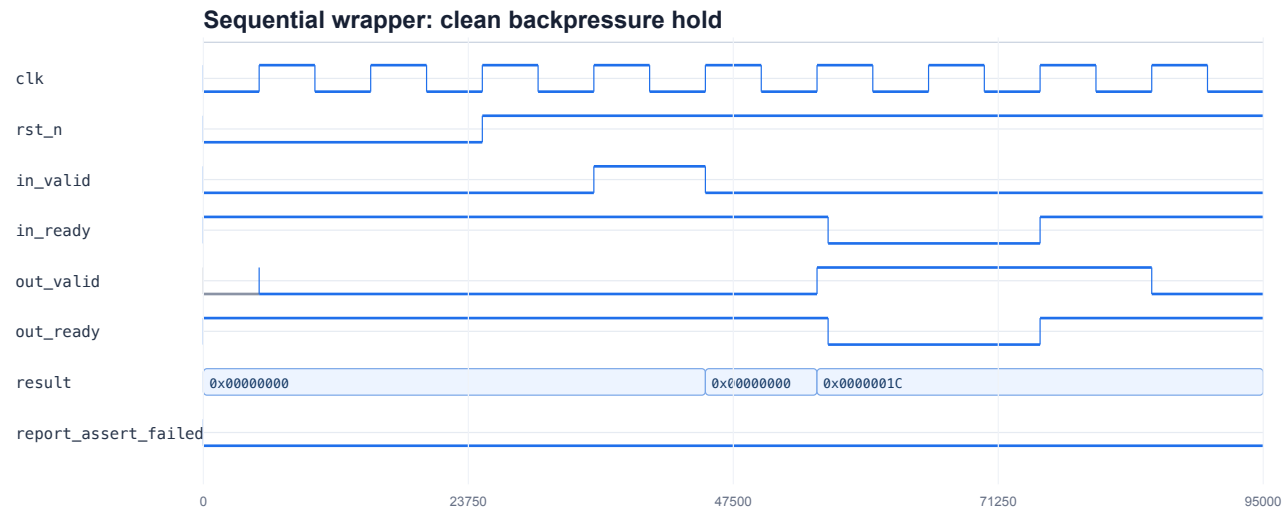


Figure 5.2: Sequential wrapper waveform: clean backpressure hold

5.5 Waveform: Bug-Injected Assertion Failure

Figure 5.3 shows the bug-injected wrapper dropping out_valid under backpressure. This directly violates the p_hold_stable contract and raises the report assertion failure flag.

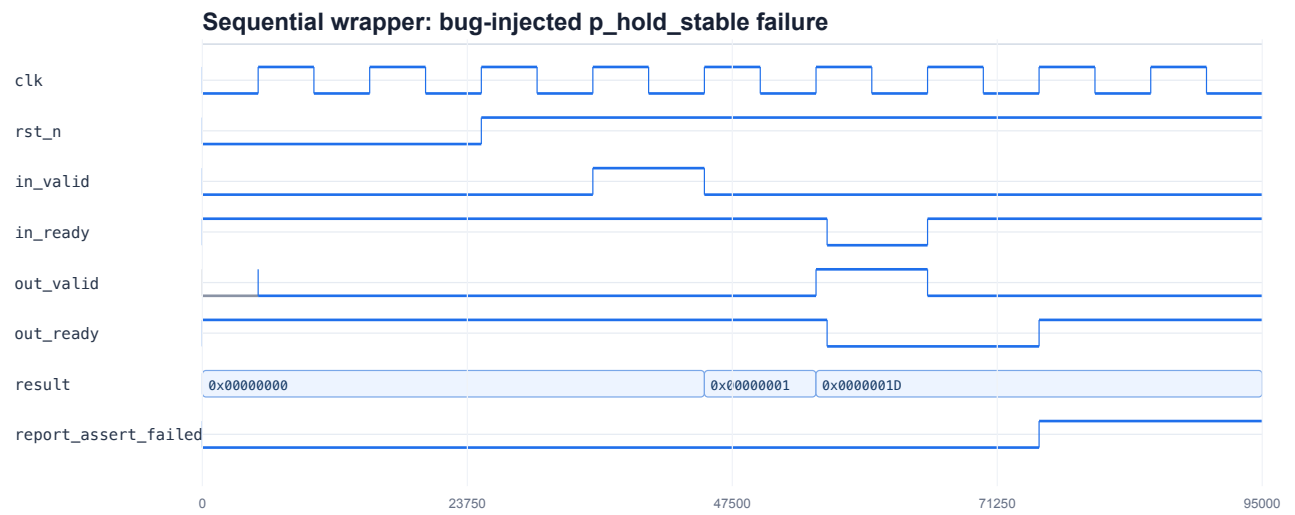


Figure 5.3: Sequential wrapper waveform: bug-injected p_hold_stable failure

6 Bug-Injection Traceability

Table 6.1: Seeded bugs and detection points

Tier	Seeded fault	Catching property/check	Outcome
INT8	LSB corruption in final rounding	a_result_matches_ref	Falsified
INT8 wrapper	Drop held output under stall	p_hold_stable	Falsified
BF16	Subnormal no longer flushed to zero	Lane decode/product assertions	Falsified
BF16	Alignment off by one	a_align_lane	Falsified
BF16	Truncate instead of RNE	a_numeric_round	Falsified
BF16	Inf-minus-Inf dropped	Top special-ladder assertions	Falsified
NVFP4	E2M1 6.0 decoded incorrectly	Lane decode/product assertions	Falsified
NVFP4	UE4M3 exponent bias off by one	a_exp	Falsified
NVFP4	NaN scale detection threshold changed	a_scale_nan_ref	Falsified
NVFP4	NaN bypass disabled in final round	a_nan_bypass	Falsified

7 Conclusion

The project demonstrates a complete verification story for a compact but high-risk mixed-precision arithmetic block. The final evidence supports these conclusions:

- The INT8 datapath is exhaustively equivalent to the golden reference.
- The sequential wrapper satisfies ready/valid stability, reset, stall, and status-alias properties.
- The BF16 tier is proven by decomposing nonlinear logic into tractable standalone proofs and a top-level pre-round assume-guarantee proof.
- The NVFP4 tier is proven using lane, scale, round, and pure-DUT-net top properties that avoid expensive golden multiply elaboration in the top miter.
- The UVM regression validates integrated behavior across all supported modes with zero scoreboard residue or mismatches.
- Bug-injection runs confirm the proof suite has teeth: every targeted mutation is caught.
- Reachable coverage closes to 100.00%; the remaining waiver is structurally justified.

The resulting methodology is reusable: keep golden references shared, isolate nonlinear blocks, prove protocol separately from arithmetic, and pair every clean proof family with a meaningful failing mutation.

A Reproduction Commands

The report artifacts are generated with the following commands from the repo root, assuming compatible EDA tools are available in the shell environment:

Listing A.1: Report evidence and PDF build commands

```
bash doc/final_report/scripts/run_report_regression.sh
vsim -c -do doc/final_report/scripts/run_report_waveforms.do
bash doc/final_report/scripts/build_assets.sh
bash doc/final_report/scripts/build_report.sh
```

Raw tool logs, waveform databases, UCDB files, and transcripts are generated artifacts. They are not checked into the repository; this report includes summarized evidence and curated figures only.